

ATV · RU Digital v2 — da especificação ao padrão no código

Roteiro prático · atividade em grupo, feita inteiramente em casa · 1 semana · Java

Nome	Matrícula	Assinatura

Antes de começar

Esta atividade é feita **inteiramente em casa, sem aula de laboratório e sem orientação do professor**. Ela foi desenhada para isso: tudo o que vocês precisam está em três lugares — neste **roteiro prático**, no **roteiro conceitual** (a aula, em cinco blocos: A a E) e no **esqueleto de código ru-digital-java**. Quando dois deles parecerem discordar, **os testes decidem**.

O QUE VOCÊS VÃO FAZER

Atender quatro **pedidos de mudança** num sistema que já funciona — o RU Digital —, cada um com um requisito com medida. Para cada pedido: **decidir** (um ADR), **desenhar** (C4 nível 3), **refatorar** o código até os testes de aceitação ficarem verdes e **provar** a fronteira com um teste. Cada pedido pede um princípio de projeto, e cada princípio tem a forma de um padrão da aula 06:

Pedido	O que muda	Padrão	Quanta ajuda
1	trocar o gateway de pagamento (RNF5)	Adaptador + Fábrica	Ciclo 1 · andaime completo
2	nova regra de entrada (RNF9)	Strategy	Ciclo 2 · andaime médio
3	bolsista titular, uma vez por refeição (RNF8)	Decorador	Ciclo 3 · andaime leve
4	avisar sobre o cardápio (RNF10)	Observador	Ciclo 4 · sem andaime

O **andaime** diminui de propósito: no Ciclo 1 o roteiro diz quase cada passo; no Ciclo 4, só o requisito e o critério de pronto. É o mesmo método quatro vezes, com cada vez menos apoio — no fim, vocês o fazem sozinhos.

Como cada ciclo funciona

Objetivo	o que vocês devem ter produzido no fim do ciclo, em uma frase
Leia	os blocos do roteiro conceitual que dão a base — leiam antes de fazer
Faça	passos numerados, com tempo sugerido (é uma estimativa, não um limite)
Confira	a lista que o professor usa para corrigir. Não avancem com item em aberto
Travou?	dicas no Apêndice A, em dois níveis. A Dica 1 aponta a direção; a Dica 2 quase entrega. Usem nessa ordem, e só depois de tentar

Regras

- Grupo de 2 ou 3 pessoas, o mesmo do começo ao fim. Uma entrega por grupo. **Todos** devem conseguir explicar qualquer parte da entrega.
- Não editem** nada em `test/` exceto `TestesFronteira.java` (lá vocês **acrescentam** regras), nem nada em `src/ru/gateways/`, `src/ru/app/`, `src/ru/rede/` e `ferramentas/`. Na correção, esses

arquivos são **restaurados** a partir do original: mudanças neles não contam, e a edição é registrada.

- **Não mudem a assinatura** de `Montagem.criarCatraca`, `MontagemCreditos.criarServicoCompra` e `MontagemCardapio.criar`: é por eles que os testes entram. O conteúdo desses métodos é de vocês.
- Podem criar quantas classes, interfaces e pacotes quiserem dentro de `src/ru/`. Os testes **não exigem nomes** de classes.
- Rodem os testes depois de cada passo pequeno. Refatorar é mudar a estrutura **sem mudar o comportamento** — e quem diz se o comportamento mudou é a suíte `funcionais`.
- Uso de assistentes de IA: podem consultar, mas o texto dos ADRs e a reflexão devem ser de vocês, e qualquer membro do grupo pode ser chamado a explicar uma decisão ou um trecho de código.

Cronograma sugerido

Quando	Carga	O que
Dia 1	≈ 3 h	Leitura do roteiro conceitual (≈ 2 h, individual, antes de o grupo se reunir) · Preparação (Etapa 0)
Dias 2 e 3	≈ 3 h	Ciclo 1 — com calma e com o grupo junto
Dia 4	≈ 2 h	Ciclo 2
Dia 5	≈ 3 h	Ciclo 3
Dia 6	≈ 2 h	Ciclo 4
Dia 7	≈ 1 h	Fechamento · entrega no Moodle

A semana é curta: são **≈ 14 h de trabalho por grupo**, cerca de 2 h por dia. **Não deixem para os últimos dias** — cada ciclo parte do código do anterior, e o Ciclo 1 é o mais longo.

Façam o Ciclo 1 com calma e **juntos**: ele ensina o método que os outros três repetem. Uma forma que funciona: uma pessoa digita, as outras leem o roteiro e conferem; troquem a cada ciclo.

Como trabalhar em grupo à distância

- Uma **única cópia oficial** do projeto (um repositório Git privado, ou uma pasta compartilhada). Duas cópias editadas em paralelo viram conflito.
- Reuniões curtas por chamada com tela compartilhada funcionam bem para os passos de refatoração.
- Antes de parar, rodem `rodar testes` e anotem onde pararam (quem retomar começa do verde).

Ambiente e diagnóstico

OBJETIVO

O esqueleto rodando no computador de **cada** integrante, e o grupo sabendo o que está verde, o que está vermelho e por quê.

Faça — instalação (cada integrante)

1. Abram um terminal e rodem `javac -version`. Se aparecer `javac 17` ou mais novo (21, 25...), pulem para o passo 4.
2. Instalem um **JDK** (não basta o JRE): o **Eclipse Temurin 21 (LTS)**, em adoptium.net, é gratuito e tem instalador para Windows, macOS e Linux. No Windows, no instalador, marquem **Add to PATH** e **Set JAVA_HOME**. No Linux, o pacote da distribuição também serve (ex.: `sudo apt install openjdk-21-jdk`).
3. **Fechem e abram de novo** o terminal e repitam `javac -version`.
4. Baixem `ru-digital-java.zip` do Moodle, extraiam e abram um terminal **dentro da pasta** `ru-digital-java` (a que tem o `README.md`). No Windows: na pasta, cliquem na barra de endereço, digitem `cmd` e Enter.

WINDOWS × LINUX/MACOS

Neste roteiro os comandos aparecem como `rodar testes`, que é a forma do **Windows**. No **Linux e no macOS**, troquem `rodar` por `./rodar.sh` — por exemplo, `./rodar.sh testes`. Se aparecer `*Permission denied*`, usem `sh rodar.sh testes`.

Atenção ao PowerShell (o terminal padrão do Windows 10/11 e do VS Code; o prompt começa com `PS`): ele não executa scripts da pasta atual pelo nome. Lá, escrevam `.\rodar testes` (com `.\` na frente) — ou abram o **Prompt de Comando** (`cmd`), onde `rodar testes` funciona como está.

Faça — diagnóstico (o grupo junto)

1. Rodem `rodar testes`. O script compila tudo e roda as seis suítes. O esperado é terminar com `Rodados 36 testes: 20 ok, 9 falhas, 7 erros`. Isso é **normal**: os vermelhos são o trabalho de vocês.
2. Leiam o resumo do topo. Deve estar assim:

```
funcionais 10 testes · OK
fronteira  2 testes · OK
pedido1    7 testes · 6 não passaram
pedido2    5 testes · 4 não passaram
pedido3    9 testes · 3 não passaram
pedido4    3 testes · 3 não passaram
```

1. Rodem a simulação de um almoço sem rede: `rodar simular --sem-rede`. Olhem as linhas da Carla e do Diego:

```
Carla (bolsista), QR do próprio app      BLOQUEIA  credencial não aceita
Diego com a carteirinha EMPRESTADA da Carla LIBERA    ok
Diego de novo, 1 min depois              LIBERA    ok
```

A catraca decide sem rede — o RNF2 está cumprido. Mas a Carla, que é bolsista, não consegue entrar com o QR do próprio app, e o Diego almoça de graça duas vezes com a carteirinha dela. Guardem isso: é o pedido 3.

1. Rodem `rodar grafo ru.creditos` e procurem as linhas marcadas com `<-- sai do pacote`. Guardem também: é o pedido 1.
2. Abram o projeto no editor que preferirem (VS Code com o `*Extension Pack for Java*`, IntelliJ IDEA Community ou Eclipse). Marquem `src`, `test` e `ferramentas` como pastas de código-fonte. O editor ajuda a navegar, mas **confiram sempre pelo script**, que é como a correção roda.

3. Leiam o [README.md](#) do esqueleto e o [docs/decisoões/ADR-001-autorizacao-na-borda.md](#) (é o exemplo resolvido).

CONFIRA

- ☐ `javac -version` mostra 17 ou mais novo em todos os computadores do grupo.
- ☐ `rodar testes` termina em `Rodados 36 testes: 20 ok, 9 falhas, 7 erros`.
- ☐ `rodar simular --sem-rede` mostra a Carla bloqueada com o próprio QR e o Diego liberado com a carteirinha dela.
- ☐ Todos leram o roteiro conceitual até o fim, pelo menos uma vez.

Travou? Apêndice A, **Etapas 0**. Problema de instalação ou compilação? Apêndice B.

Trocar o gateway de pagamento

0 REQUISITO

RNF5 · Trocar o gateway de pagamento (novo contrato) altera **um único arquivo** da API de Créditos, **sem tocar nas regras de crédito**, e os testes existentes continuam passando. O novo fornecedor é a **CobraFácil** (<src/ru/gateways/cobrafacil/>).

OBJETIVO

ADR-002 escrito; nível 3 da API de Créditos desenhado; uma regra de fronteira que falhou antes e passa depois; [pedido1](#) verde; a troca do gateway padrão feita alterando **um** arquivo.

LEIA

- Roteiro conceitual: **Blocos B, C e D** inteiros e **E.1 a E.3** (Adaptador e Fábrica).
- Código: <src/ru/creditos/ServicoCompra.java>, [MontagemCreditos.java](#) e os dois SDKs em <src/ru/gateways/> — reparem como a CobraFácil é diferente da PagaPix (valor, retorno, exceção).
- <test/ru/testes/TestesPedido1.java>: é ele que define “pronto”. Leiam o que cada teste exige.

1.1 · Decisão (40 min)

1. **Alternativa A** — “o mínimo”: um `if (gateway.equals("cobrafacil"))` dentro do [ServicoCompra](#). Descrevam em duas linhas o que ela exige mudar hoje e a cada fornecedor novo.
2. **Alternativa B** — a do Bloco E.3: a regra de crédito depende de um contrato **do RU**; um adaptador por fornecedor; uma fábrica escolhe qual. Descrevam em duas linhas, com os nomes que vocês pretendem usar.
3. **Matriz**. Avaliem A e B contra os cenários abaixo. Marquem com • as linhas em que as colunas discordam: são os trade-offs.

Cenário	A · if no ServicoCompra	B · contrato + adaptadores	Trade-off
RNF5 · troca de gateway em um único arquivo			
RNF1 · compra em ≤ 2 s no pico			
RNF3 · toda alteração de saldo auditada			
Testar a regra de crédito sem o SDK			
Quantidade de classes e de conceitos			

1. **ADR-002**. Copiem docs/decisoess/_MODELO.md para <docs/decisoess/ADR-002-<nome-curto>.md> e preencham contexto (com a matriz), ponto de sensibilidade, ponto de trade-off, decisão, consequências e alternativa descartada. Na decisão, descrevam **a estrutura** e só então nomeiem os padrões (Bloco B.5). *Estrutura que nasce* e *Evidência* vocês completam em 1.2 e 1.3.

1.2 · Estrutura (20 min)

1. **Nível 2**. Na figura de contêineres (roteiro conceitual, C.3), qual contêiner a decisão toca? Ela cria contêiner novo? Escrevam a resposta, com o porquê, em *Estrutura que nasce* no ADR-002.
2. **Nível 3**. Em <docs/c4-creditos.md>, a primeira figura é o estado atual. Na segunda, desenhem em Mermaid como fica depois: os componentes existentes, a interface, os adaptadores, a fábrica

e os dois SDKs, com as setas de dependência (de quem usa para quem é usado; `- . implementa . ->` para implementação).

3. Em cada caixa nova, entre colchetes, o cenário da especificação que a justifica.

Para ver o Mermaid: cole o bloco em **mermaid.live**, ou usem o VS Code com a extensão `*Markdown Preview Mermaid Support*`.

1.3 · A fronteira no código (20 min)

1. Rodem `rodar grafo ru.creditos` e comparem com o diagrama de 1.2: **onde o código contradiz o desenho?**
2. Em `test/ru/testes/TestesFronteira.java`, na seção `*CICLO 1*`, escrevam um método `public void test...()` com a regra que prova o ADR-002, usando `Fronteiras.naoPodeUsar` (Bloco D.5). Pergunta-guia: dentro de `ru.creditos`, **quem** pode usar `ru.gateway`s?
3. Rodem `rodar testes fronteira`. **A regra nova precisa falhar agora**, apontando `ServicoCompra -> ru.gateway`s... Se passou, ela não está vigiando o que devia.

1.4 · Refatoração em passos pequenos (60 min)

Depois de **cada** passo, rodem `rodar testes funcionais`. Se ficar vermelho, desfaçam o último passo — não empilhem erro.

1. **Contrato.** Criem um subpacote (ex.: `src/ru/creditos/pagamento/`) com a interface que a regra de crédito vai usar, no vocabulário do RU — ex.: `Pagamento`, com um método que cobra um valor (`BigDecimal`) de uma matrícula com um token — e um tipo de resultado (ex.: um `record` com `*aprovado*`, `*transação*` e `*motivo*`). Lembrem: a primeira linha de cada arquivo é `package ru.creditos.pagamento;` e o nome do arquivo é o nome da classe pública.
2. **Adaptador PagaPix.** Criem a implementação do contrato para a PagaPix e **movam para ela** tudo o que hoje, em `ServicoCompra`, fala a língua da PagaPix: centavos, `createCharge`, `Status.PAID`, `DECLINED`, `chargeId`, `declineReason`. Mantenham o motivo de recusa que o `ServicoCompra` monta hoje (`recusado: ...` ou `falha no gateway: ...`): ele vai para a trilha de auditoria (RNF3).
3. **Injeção.** Façam `ServicoCompra` receber o contrato pelo **construtor**, em vez de criar o cliente da PagaPix. Ajustem `MontagemCreditos` para criar o adaptador e passá-lo. Rodem `funcionais`: tem que continuar verde.
4. **Adaptador CobraFácil.** Criem a segunda implementação. Atenção: o SDK lança `CobraFacilErro` (exceção verificada) em falha técnica — o adaptador a **captura e traduz** para um resultado não aprovado. Os tokens de teste são `tok_ok`, `tok_recusado` e `tok_erro`.
5. **Fábrica.** Criem uma fábrica simples que recebe o nome do gateway (`"pagapix"`, `"cobrafacil"`) e devolve o contrato. Façam `MontagemCreditos.criarServicoCompra(gateway, ...)` usá-la no lugar do `if` atual.
6. Rodem `rodar testes pedido1` e `rodar testes fronteira` até ficarem verdes. Depois, `rodar testes` completo.

1.5 · A prova do “um único arquivo” (10 min)

1. Com tudo verde, rodem `rodar diff --marcar`.
2. Façam a troca efetiva: o gateway **padrão** do sistema (`GATEWAY_PADRAO`) passa a ser a CobraFácil.
3. Rodem `rodar diff --desde-marco`. Deve aparecer **um** arquivo alterado. Rodem `rodar testes` de novo. **Guardem essa saída** para o `ENTREGA.md`.

1.6 · Segredos (15 min)

Completem a seção `ru.creditos` de `docs/SEGREDOS.md`: para cada classe (as antigas e as novas), o segredo que ela esconde e a mudança que ela absorve. Completem também `*Evidência*` no ADR-002, citando o teste que vocês escreveram.

CONFIRA

☐ `rodar testes funcionais, fronteira e pedido1` → OK.

- ☐ A regra de fronteira de vocês **falhou antes** da refatoração (vocês viram o vermelho).
- ☐ `rodar diff --desde-marco` mostrou **um único** arquivo alterado na troca do gateway padrão.
- ☐ `rodar grafo ru.creditos`: nenhuma seta de `ServicoCompra` para `ru.gateways`.
- ☐ ADR-002: a decisão é uma **estrutura** (padrão nomeado depois); o contexto cita o RNF5 com a medida; a linha **Paga** não está vazia; há alternativa descartada, com motivo.
- ☐ `c4-creditos.md`: a caixa da regra de crédito não tem seta para nenhum SDK; cada caixa nova cita um cenário.
- ☐ Em `SEGREDOS.md`, nenhum segredo é uma etapa do fluxo (“recebe o pedido” não é segredo).

Travou? Apêndice A, **Ciclo 1**. Problema de instalação ou compilação? Apêndice B.

A regra de entrada muda por decisão do conselho

O REQUISITO

RNF9 · A regra de entrada é escolhida na montagem da catraca, a partir da configuração: **SALDO** (entra quem tem saldo $\geq$ tarifa) ou **TOLERA_NEGATIVO** (entra quem fica com saldo $\geq$ -R\$ 5,00 depois de pagar a tarifa). **Incluir uma regra nova não altera a classe `Catraca`.**

OBJETIVO

ADR-003 escrito; **Catraca** livre de regras de entrada; as duas regras funcionando; **pedido2** verde.

LEIA

- Roteiro conceitual: **D.1 a D.3** (de novo, agora olhando a **Catraca**) e **E.4** (Strategy).
- `src/ru/catraca/Catraca.java` e `Montagem.java`; `test/ru/testes/TestesPedido2.java`, inclusive os valores de saldo da Ana, Bia, Duda e Caio.

Faça

1. **ADR-003** (30 min). Avaliem pelo menos duas alternativas — uma delas é “mais um **case** no **switch**” — contra RNF9, RNF2 (decisão em ≤ 1 s sem rede) e testabilidade. Declarem o ponto de trade-off. Mesmo molde do ADR-002.
2. **Estrutura** (15 min). Em `docs/c4-catraca.md`, na seção *Depois dos pedidos 2 e 3*, comecem o diagrama: a **Catraca**, a abstração da regra e as regras concretas.
3. **Código** (60 min). Tirem a regra de dentro da **Catraca**: a **Catraca** passa a **receber** a regra pronta no construtor e só a chama; cada regra vira uma classe; o texto da configuração ("**SALDO**", "**TOLERA_NEGATIVO**") é traduzido em objeto **só** em `Montagem.criarCatraca`. Façam em passos: primeiro só **SALDO** (rodem `funcionais`), depois **TOLERA_NEGATIVO** (rodem `pedido2`). Uma regra que recebe um valor configurável (o limite de -R\$ 5,00) o recebe **pelo construtor**.
4. **Segredos** (15 min). Comecem a seção `*ru.catraca` depois dos pedidos 2 e 3* em `SEGREDOS.md`.

CONFIRA

- ☐ rodar testes → `funcionais`, `fronteira`, `pedido1` e `pedido2` **OK**.
- ☐ O código da **Catraca** não tem `switch`, nem o texto "**SALDO**", nem `compareTo` (o próprio `pedido2` confere).
- ☐ Um valor de configuração inválido é recusado **na montagem**, não na hora em que alguém passa na catraca.
- ☐ ADR-003: a linha **Paga** diz o que piorou; o nível do C4 em que a decisão aparece está justificado.
- ☐ Vocês conseguem explicar, em uma frase, qual princípio do Bloco D.3 o pedido 2 exigiu.

Travou? Apêndice A, **Ciclo 2**. Problema de instalação ou compilação? Apêndice B.

O bolsista e a carteirinha emprestada

O REQUISITO

RNF8 · A isenção do bolsista integral só pode ser usada pelo titular: carteirinha emprestada ou credencial copiada não autoriza a entrada, e cada bolsista passa no máximo uma vez por refeição. **Vale também com o enlace caído**, sem violar os tempos do RNF2.

OBJETIVO

ADR-004 com três alternativas; o nível 3 da catraca completo; `pedido3` verde sem tocar na `Catraca`; a simulação mostrando o Diego barrado.

Agora com menos ajuda. O pedido tem **duas perguntas** separadas — **quem é a pessoa?** e **ela já passou nesta refeição?** —, e cada uma tem seu segredo.

LEIA

- Roteiro conceitual: **A.5** e **A.6** (de onde veio o RNF8; quais características da ISO ele envolve), **C.4**, **D.2** e **E.5** (Decorador).
- `src/ru/app/QRCode.java` — o app já gera um QR dinâmico assinado. O comentário explica o formato e como verificá-lo **sem rede**.
- `src/ru/catraca/Replica.java` (`chaveQr`), `Refeicoes.java` e `Relogio.java`.
- `test/ru/testes/TestesPedido3.java` — inclusive o último teste: ele diz algo sobre **onde** a solução pode entrar.
- A LGPD, art. 5º, II, e art. 11 (link nas referências do roteiro conceitual).

Faça

1. **ADR-004** (60 min). Avaliem **pelo menos três** alternativas para garantir que quem passa é o titular — entre elas **foto do titular exibida ao operador**, **QR dinâmico no app** e **biometria facial** — contra: RNF8, RNF2 (sem rede, ≤ 1 s), RNF6 e LGPD, inclusão (quem não tem celular?) e custo. A biometria é uma alternativa **válida**; se o grupo a escolher, registre o que ela exige. Declarem o ponto de trade-off e o **que acontece com duas catracas sem enlace**.
2. **Estrutura** (20 min). Completem `docs/c4-catraca.md` com os componentes novos, anotados.
3. **Código** (80 min). Façam `pedido3` ficar verde **sem editar a `Catraca`** e mantendo `fronteira` e `pedido2` verdes. As peças novas entram pela `Montagem`. Os testes usam o QR dinâmico porque é o que um protótipo consegue verificar; se o ADR escolher outra alternativa, implementem o QR e registrem no ADR como a estrutura permitiria trocar depois.
4. **Prova** (5 min). `rodar simular --sem-rede`: confirmam as linhas da Carla e do Diego. Guardem a saída para o `ENTREGA.md`.
5. **Segredos** (15 min). Terminem a seção da catraca em `SEGREDOS.md`.

CONFIRA

- ☐ `rodar testes` → tudo OK até `pedido3`.
- ☐ A `Catraca` não foi editada neste ciclo (e não menciona bolsista, QR, HMAC nem refeição).
- ☐ Na simulação: Carla **LIBERA** com o próprio QR; Diego **BLOQUEIA** com a carteirinha dela; Carla **BLOQUEIA** na segunda vez na mesma refeição.
- ☐ ADR-004 compara três alternativas, e a linha **Paga** fala de pelo menos uma destas: LGPD, inclusão, duas catracas sem enlace.
- ☐ Vocês conseguem apontar, no código, qual peça é um **Decorador** e o que ela envolve.

Travou? Apêndice A, **Ciclo 3**. Problema de instalação ou compilação? Apêndice B.

Avisar sobre o cardápio

O REQUISITO

RNF10 · Quando a semana é publicada, app, painel e e-mail da gestão são avisados. Um novo interessado (ex.: o bot do DCE) pode ser acrescentado **sem alterar a classe** ``Cardapio``, que **não depende de nenhum canal de aviso**.

Sem passos desta vez. Vocês têm o método (quatro vezes aplicado), o Bloco **E.6** e os testes: `pedido4` define “pronto”. Leiam `src/ru/cardapio/` e `src/ru/avisos/`, e decidam.

CONFIRA

- ☐ `docs/decisoes/ADR-005-...md` completo, com alternativa descartada e linha **Paga** (pensem: e se um canal falhar?).
- ☐ `docs/c4-cardapio.md` com o nível 3 depois da decisão.
- ☐ Uma regra de fronteira nova em `TestesFronteira.java` que prova o ADR-005 — e que falhou antes da refatoração.
- ☐ `funcionais` continua verde: os três canais atuais seguem avisados, exatamente uma vez.
- ☐ `rodar testes` → **0 falhas, 0 erros** em todas as suítes.
- ☐ Seção do cardápio completa em `SEGREDOS.md`.

Travou? Apêndice A, **Ciclo 4**. Problema de instalação ou compilação? Apêndice B.

Entrega

O quê	A pasta ru-digital-java inteira, compactada em .zip (sem a pasta out/), com um arquivo ENTREGA.md na raiz.
Onde	Moodle, na tarefa *ATV · RU Digital v2 — da especificação ao padrão no código*. Uma entrega por grupo.
Prazo	_____ (definido no Moodle)
Nota	Atividade com nota própria.

O arquivo ENTREGA.md

```
# Entrega · RU Digital v2
Grupo: <nome – matrícula>, <nome – matrícula>, <nome – matrícula>

## Saída de rodar testes (só o resumo do topo e a última linha)
<colem aqui>

## Saída de rodar simular --sem-rede
<colem aqui>

## Saída de rodar diff --desde-marco (na troca do gateway padrão)
<colem aqui>

## Quem fez o quê (uma linha por integrante)

## Reflexão (até 12 linhas)
1. Nos quatro pedidos, a mudança entrou por código novo ou por edição?
   Onde houve edição, por que ela foi inevitável?
2. Qual padrão teria sido cerimônia se o pedido não existisse? Por quê?
3. Que decisão do ADR-001, tomada por causa do RNF2, tornou possível
   atender o RNF8 sem editar a Catraca?
```

Como será avaliado

Critério	Insuficiente	Adequado	Bom
Decisão (ADR-002 a 005)	Nome de padrão ou tecnologia no lugar da decisão, ou ADR faltando	Contexto e decisão ligados a um cenário da v2	Alternativas descartadas + consequência com o que piorou (no RNF8: LGPD, inclusão ou duas catracas)
Estrutura (três c4-*.md)	Diagrama desligado do código	Componentes e setas coerentes com o código	Cada componente novo anotado com o cenário que o justifica
Princípios e padrões (SEGREDOS.md, reflexão)	Classes descritas como etapas do fluxo; padrão sem problema	Um segredo por classe; padrão ligado ao pedido	Segredo + mudança que absorve + princípio; reconhece quando o padrão seria cerimônia
Evidência (testes)	Testes de aceitação vermelhos	Os quatro pedidos verdes	Tudo verde, troca em um arquivo, regras de fronteira próprias que falharam antes

Checklist final

- ☐ **rodar testes** → 0 falhas, 0 erros.
- ☐ **docs/deciso**es/ tem ADR-002, ADR-003, ADR-004 e ADR-005.
- ☐ **docs/c4-credit**os.md, **docs/c4-catra**ca.md e **docs/c4-carda**prio.md atualizados.
- ☐ **docs/SEGREDOS**.md completo.
- ☐ **TestesFronteira**.java com pelo menos **duas** regras novas (ciclos 1 e 4).

- ☐ `ENTREGA.md` na raiz, com as três saídas, a divisão do trabalho e a reflexão.
- ☐ Nenhum arquivo protegido foi editado (`test/` exceto `TestesFronteira.java`, `gateways/`, `app/`, `rede/`, `ferramentas/`).
- ☐ O `.zip` abre, e `rodar testes` funciona a partir dele numa pasta nova.

Apêndice A · Dicas progressivas

Use só quando travarem, e na ordem. Leiam a Dica 1, tentem de novo; só então a Dica 2.

Etapa 0 · Preparação

DICA 1

Se `rodar testes` não chega ao resumo, o problema é de ambiente, não de código: vejam o Apêndice B. O resumo com 9 falhas e 7 erros é o **esperado**.

DICA 2

Confiram, nesta ordem: `javac -version` mostra 17+; o terminal está na pasta que tem o `README.md` (`dir` no Windows, `ls` no Linux, deve listar `rodar.bat`, `rodar.sh`, `src...`); no Linux/macOS, `sh rodar.sh testes`.

Ciclo 1 · Pedido 1

DICA 1

Contem, em `ServicoCompra`, quantas palavras só existem por causa da PagaPix (`PAID`, `chargeId`, `centavos...`) — é o *vazamento pela porta da frente* (D.3). A decisão boa faz essas palavras sumirem da regra de crédito. Para a regra de fronteira: o terceiro parâmetro de `naoPodeUsar` é a lista de quem está **liberado** — quem, em `ru.creditos`, precisa mesmo conhecer os SDKs? Para a injeção, o modelo está na catraca: vejam como a `Catraca` recebe o `Identificador` pronto e quem o escolhe.

DICA 2

Regra: `Fronteiras.naoPodeUsar("ru.creditos", new String[]{"ru.gateway"}, "ru.creditos.pagamento");` — ela libera o subpacote dos adaptadores inteiro. Estrutura possível: `ru.creditos.pagamento.Pagamento` (interface), `ResultadoPagamento` (record com `aprovado`, `transacao`, `motivo`), `PagaPixAdapter`, `CobraFacilAdapter`, `FabricaPagamento`; `new ServicoCompra(saldos, auditoria, pagamento)`. Uma segunda regra útil: os adaptadores não usam `ServicoCompra`, `Saldos` nem `Auditoria`. No 1.5, a troca é uma linha: `GATEWAY_PADRAO = "cobrafacil"`.

Ciclo 2 · Pedido 2

DICA 1

O `switch` da `Catraca` responde a uma pergunta — “esta pessoa pode entrar pagando esta tarifa?”. Transformem a pergunta num contrato (uma interface com um método) e cada `case` numa classe. Quem hoje recebe a `String regra`? Quem deveria receber?

DICA 2

```
interface RegraEntrada { Decisao avaliar(Usuario u, BigDecimal tarifa); };
SaldoSuficiente (saldo ≥ tarifa); TolerarSaldoNegativo(BigDecimal limite) (saldo - tarifa ≥ -limite). Na Catraca, o campo String regra vira RegraEntrada regra, e o trecho do switch vira
Decisao d = regra.avaliar(u, tarifa); if (!d.liberar()) return d; Na Montagem:
switch (regra) { case "SALDO" -> new SaldoSuficiente(); case "TOLERA_NEGATIVO" ->
new TolerarSaldoNegativo(new BigDecimal("5.00")); default -> throw new
IllegalArgumentException(...); } — esse switch pode ficar: a montagem é o lugar dele.
```

Ciclo 3 · Pedido 3

DICA 1

Duas peças, cada uma com o **mesmo contrato** de uma peça que já existe: uma para *quem é a pessoa* (o contrato `Identificador`) e outra para *já passou nesta refeição?* (o contrato de regra do Ciclo 2). Cada peça nova **envolve** a antiga e delega a ela o que não é seu. O último teste diz que a `Catraca` não pode saber de nada disso; então a solução entra por onde? (D.2).

DICA 2

Identificação: `IdentificadorComTitularidade(Identificador base, Replica replica, Relogio relógio)` — se a credencial é do tipo `qr`, separa `matricula:instante:assinatura`, recalcula `QrCode.assinar(matricula, instante, replica.chaveQr(matricula))`, compara com `MessageDigest.isEqual` e aceita se `|agora - instante| ≤ 30 s` (o `agora` vem de `relogio.agora().toEpochSecond(ZoneOffset.UTC)`); se for carteirinha, delega à `base` e recusa quando o usuário é bolsista. Repetição: `UmaPorRefeicao(RegraEntrada interna, Relogio relógio)` — chama a interna; se liberou e o usuário é bolsista, guarda `matricula|data|refeição` num `Set` e bloqueia se já estava lá (`Refeicoes.refeicaoDe(...)` diz a refeição). Na `Montagem`, embrulhem as duas.

Ciclo 4 · Pedido 4

DICA 1

Quem precisa conhecer quem? Hoje o `Cardapio` conhece os canais. Invertam: os canais (ou alguém por eles) se **inscrevem** no `Cardapio`, que só conhece uma interface. Quem faz as inscrições dos três canais atuais? (Olhem o nome da classe de montagem.)

DICA 2

`interface ObservadorCardapio { void semanaPublicada(Semana s); }` no `Cardapio`, uma `List<ObservadorCardapio>`, um método público `assinar(ObservadorCardapio o)` e `publicar` percorrendo a lista. Em `MontagemCardapio.criar()`: `c.assinar(new AppAluno()::mostrarNotificacao)`; e o mesmo para `PainelWeb::atualizar` e `EmailGestao::enviarResumo` (criem cada canal numa variável antes). Regra: `Fronteiras.naoPodeUsar("ru.cardapio", new String[]{"ru.avisos"}, "ru.cardapio.MontagemCardapio")`;

Apêndice B · Problemas comuns

Sintoma	O que fazer
<code>'javac' não é reconhecido... / command not found</code>	JDK não instalado, ou fora do PATH. Reinstalem o Temurin marcando Add to PATH , fechem e abram o terminal. Só <code>java</code> funciona e <code>javac</code> não? Instalaram um JRE: precisa do JDK.
<code>release version 17 not supported / invalid target release</code>	O JDK é antigo (8 ou 11). Instalem o 17 ou mais novo e confirmem com <code>javac -version</code> .
<code>./rodar.sh: Permission denied</code>	Use <code>sh rodar.sh testes</code> (ou <code>chmod +x rodar.sh</code> uma vez).
<code>rodar</code> não é reconhecido... (Windows)	Se o prompt começa com <code>PS</code> , é o PowerShell: use <code>.\rodar testes</code> . Se não, o terminal não está na pasta do projeto: <code>cd</code> até a pasta que tem o <code>README.md</code> .
<code>class X is public, should be declared in a file named X.java</code>	Em Java, o arquivo tem o nome exato da classe pública, com maiúsculas.

Sintoma	O que fazer
<code>cannot find symbol</code> para uma classe que vocês criaram	Confiram a primeira linha: <code>package</code> tem de bater com a pasta (<code>src/ru/creditos/pagamento/</code> → <code>package ru.creditos.pagamento;</code>), e quem usa a classe precisa do <code>import</code> . A classe precisa ser <code>public</code> para ser usada de outro pacote.
<code>unreported exception CobraFacilErro; must be caught or declared to be thrown</code>	É uma exceção verificada: o adaptador a captura (<code>try ... catch (CobraFacilErro e)</code>) e traduz para um resultado não aprovado.
A IDE mostra erros, mas o script compila (ou o contrário)	Marquem <code>src</code> , <code>test</code> e <code>ferramentas</code> como pastas de código-fonte. Na dúvida, vale o script.
Testes funcionais quebraram na refatoração	Desfaçam o último passo (Ctrl+Z no editor, ou <code>git checkout</code> se usam Git) e deem um passo menor. Leiam a mensagem: ela diz o valor esperado e o obtido.
A regra de fronteira nova passa logo de cara	Ela não está vigiando o que devia. Rodem <code>rodar grafo <pacote></code> e confirmem os nomes: são nomes completos, com <code>ru.</code> na frente.
<code>pedido2</code> reclama de <code>compareTo</code> na <code>Catraca</code>	A comparação de saldo é regra de entrada: pertence às classes de regra, não à <code>Catraca</code> .
Acentos estranhos no terminal	Só estética. O <code>rodar.bat</code> já troca a página de código; se persistir, <code>chcp 65001</code> antes.
<code>rodar diff --desde-marco</code> mostra mais de um arquivo	Houve outra edição em <code>src/</code> depois do marco (a ferramenta só olha os <code>.java</code> de <code>src/</code>). Rodem <code>rodar diff --marcar</code> de novo com tudo verde e refaçam só a troca do gateway padrão.